



관광 MICE 전문가 과정

05. 버전 관리 도구

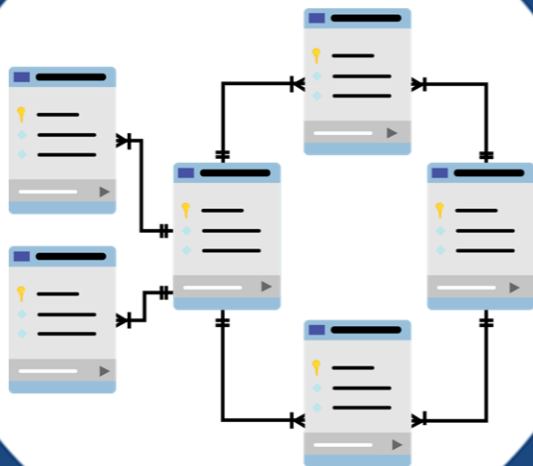
조창제

강의자료

2026.08.

목차

CONTENTS



I

코드 및 문서의 **변경 이력 관리**

버전 관리 도구

II

여러 곳에 분산 **버전 관리**

GIT

III

사용법과 **협업 흐름**

GIT의 사용





1.개념

① 버전관리도구

- 코드나 문서 파일들의 변경 이력을 기록/관리하는 소프트웨어
 - 파일을 언제, 누가, 어떻게 변경하였는지 추적
 - 특정 시점의 버전으로 되돌리기
 - 여러 사람이 동시에 작업해도 충돌을 최소화

② 종류

- GIT
 - 각자의 로컬 컴퓨터에 전체 저장소 히스토리가 복제
 - 네트워크 연결 없이도 대부분 기능 이용 가능
- SVN
 - 중앙 서버에 저장소가 존재
 - 변경 이력 조회를 하려면 항상 서버 접속 필요
 - 서버가 다운되면 협업 불가





1.개념

① git

- 분산형 버전 관리 시스템: 개발자 각자의 컴퓨터에서 전체 내역을 관리
- 프로젝트의 소스 코드 변경 이력을 기록, 관리하여 추적, 복원 가능

② git의 동작 원리

- 스테이지라는 임시 저장소에 변경 이력을 임시로 관리
- 임시로 관리된 최종 변경 이력을 하나의 버전(스냅샷)으로 기록
- 버전을 저장소에서 시간순으로 관리
 - 기록된 버전을 서로 비교 가능
 - 누가, 언제, 어떻게 변경했는지 추적 가능
 - 다양복구 방식 제공(이전 버전으로 돌아가기, 일부만 복원 등)



1. 저장소(Repository)

- 파일의 변경 이력(히스토리)를 기록, 관리하는 공간

① 구분

- 로컬 저장소: 개발자 각자의 컴퓨터에 존재하는 저장소
- 원격 저장소: github, gitlab과 같은 외부 서버(컴퓨터)에 존재하는 저장소

② 생성, 복제, 업데이트

- git init: 저장소를 생성 및 초기화 가능
- git clone <주소>: 저장소를 현재 작업 경로에 복사해올 수 있음
- git push: 로컬 저장소의 변경이력을 원격 저장소에 업데이트
- git pull: 원격 저장소의 변경이력을 로컬 저장소에 업데이트



2.스태이지(Stage)

- 변경 사항을 임시로 기록해 두는 공간
- 임시로 넣은 내용은 삭제, 추가, 복구 가능

① 상태 확인, 추가/삭제/복구

- git status
 - 스테이지 상태 확인(스테이지에 올라간 파일, 스테이지에 올라가지 않고 변경된 파일 확인 가능)
- git add <파일명>: 변경된 파일을 스테이지에 등록
- git reset <파일명>: 스테이지에 등록된 파일을 제거
- git restore <파일명>: 저장소에 기록되어 있었던 상태로 복구(수정한 내용이 삭제, **복원 불가**)



장바구니
(Stage)



결제
(Commit)



영수증 관리
(Repository)

2.스태이지(Stage)

- 변경 사항을 임시로 기록해 두는 공간
- 임시로 넣은 내용은 삭제, 추가, 복구 가능

② 저장소에 반영

- git commit: 임시로 저장된 내용을 하나의 버전(스냅샷)으로 만들어 저장소에 등록
- git commit -amend -m <새로운 메시지>: 마지막 commit 메시지 수정
- git reset --soft HEAD~n: n 단계 이전의 commit을 stage로 복원
- git reset --hard HEAD~n: n 단계 이전의 commit을 삭제(변경 내용 삭제)
- git rebase -i HEAD~3: 여러 커밋의 변경이력(history) 수정



장바구니
(Stage)



결제
(Commit)



영수증 관리
(Repository)



2. 스테이지(Stage)

- 변경 사항을 임시로 기록해 두는 공간
 - 임시로 넣은 내용은 삭제, 추가, 복구 가능
- ③ 원격 저장소 반영 취소
- `git log -n <건수> --oneline`: n 건의 git commit 메시지와 commit의 고유 번호의 개념인 HEAD 정보 출력
 - `git revert <HEAD>`:
 - 기존 기록을 남긴 채 저장소에서 HEAD에 해당하는 상태로 복원



장바구니
(Stage)



결제
(Commit)



영수증 관리
(Repository)



3. 충돌

- pull/push 충돌은 같은 파일의 같은 라인 수정 시 충돌

① 충돌 해소

- git status: 충돌 파일 both modified로 나타남
- 수동으로 수정 후 add, commit, push 로 해결



장바구니
(Stage)



결제
(Commit)



영수증 관리
(Repository)

4. 분지(Branch)와 병합(Merge)

① 저장소 분할

- 여러 기능이나 실험을 위해 저장소를 분지

② 브랜치 생성/조회

- `git branch`: 로컬 브랜치 목록 조회
- `git branch <브랜치명>`: 새로운 브랜치 생성(이동 필요)
- `git branch -d <브랜치명>`: 브랜치 삭제
- `git switch <브랜치명>`: 해당 브랜치로 이동
- `git pull origin <원격 브랜치명>`: 원격 브랜치 내용을 로컬 브랜치로 가져와 병합
- `git push origin <원격 브랜치명>`: 로컬 브랜치 내용을 원격 브랜치 내용으로 반영
- `git push origin --delete <원격 브랜치명>`: 원격 브랜치 삭제

③ 저장소 병합

- 두 브랜치의 변경 사항을 하나로 합치는 작업
- `git switch <브랜치1>` 후 `git merge <브랜치2>`: 브랜치1에 브랜치2 내용을 복제(브랜치2는 유지)



1. Git 기본

1) 최초 설정

- ① git config: Git 사용 환경을 적절히 설정
 - git config --global user.name "aaa"
 - git config --global user.email "aaa@email.com"
 - git config -list (나갈 때는 q 누를 것)

2) 가져오기

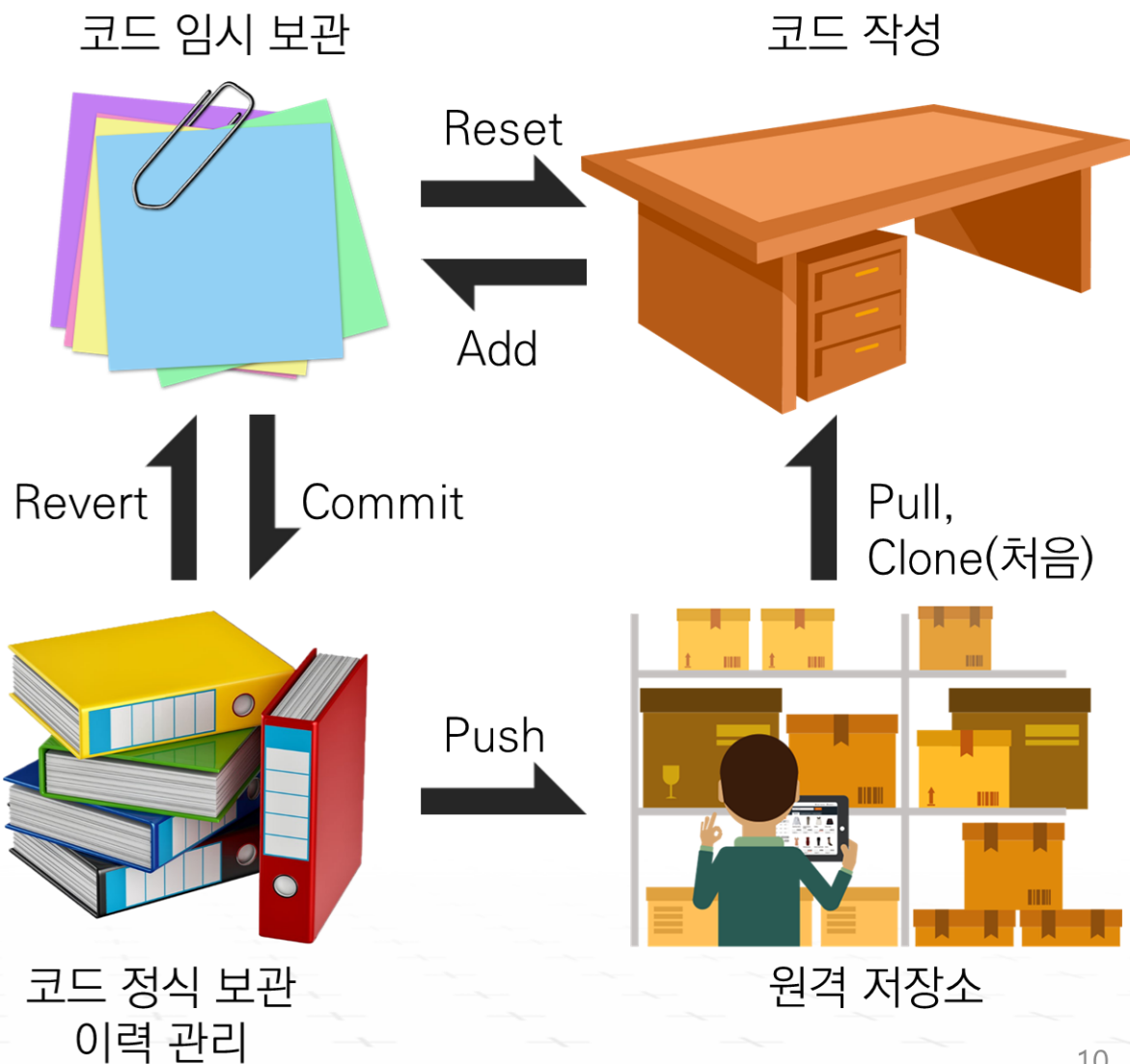
- ① git clone <URL> <폴더명>

3) 파일 관리 목록 추가

- ① git add <추가할 파일>, git status - 상태확인

4) 변경 작업을 저장소에 기록

- ① git commit -m "메세지명"





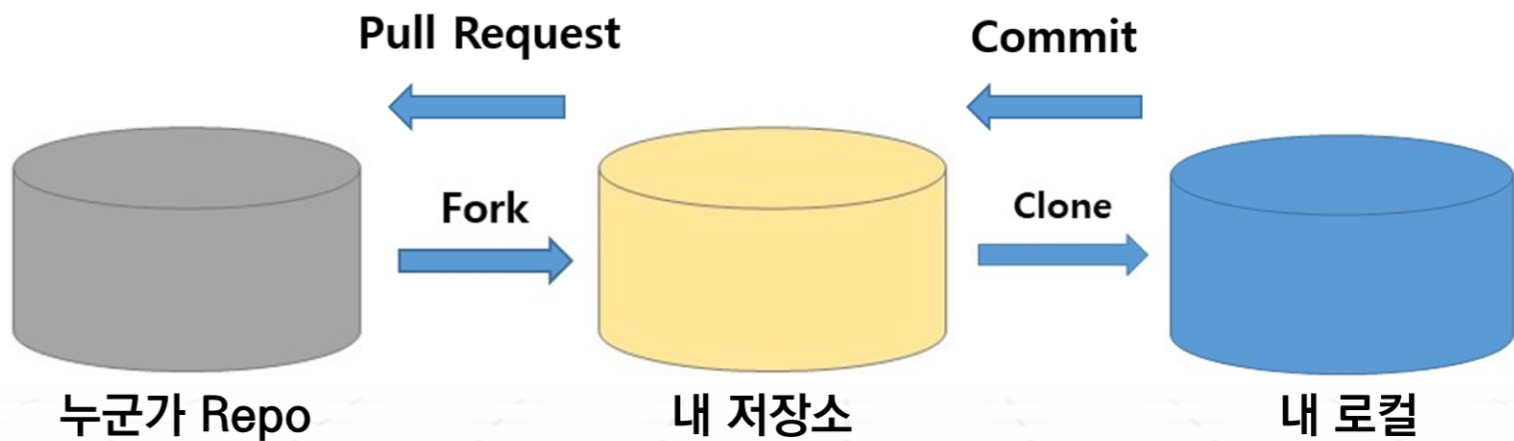
1. Git 같이 쓰기

1) Fork

- ① 다른 사람의 저장소에 있는 레포지토리를 내 원격 저장소로 가져오는 것
 - 다른 사람의 프로젝트에 참여하여 기여자로 공헌하고 싶을 때
 - 즉, 남의 코드를 가져와서 수정하고, 다시 원 주인에게 수정 요청할 때

2) Pull Request

- ① 내가 수정한 코드가 있으니 내 branch를 가져가 검토 후 병합해 달라고 요청하는 것

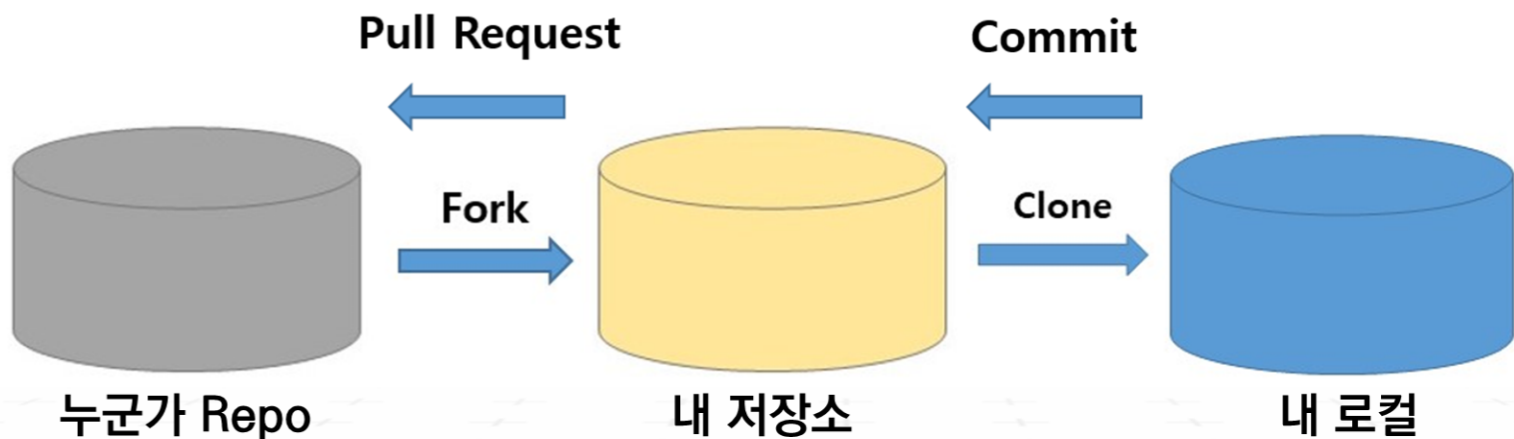




1. Git 같이 쓰기

1) Pull Request 과정

- ① 원본 저장소 fork하기 > Fork한 저장소 로컬로 Clone 하기 > 브랜치 생성 > 브랜치 이동
 - git branch [브랜치 조회]
 - git branch <branch명> [브랜치 생성]
 - git switch <branch명> [브랜치 이동]
- ② 수정 후 push(Add/Commit/Push)
- ③ Compare & Pull request

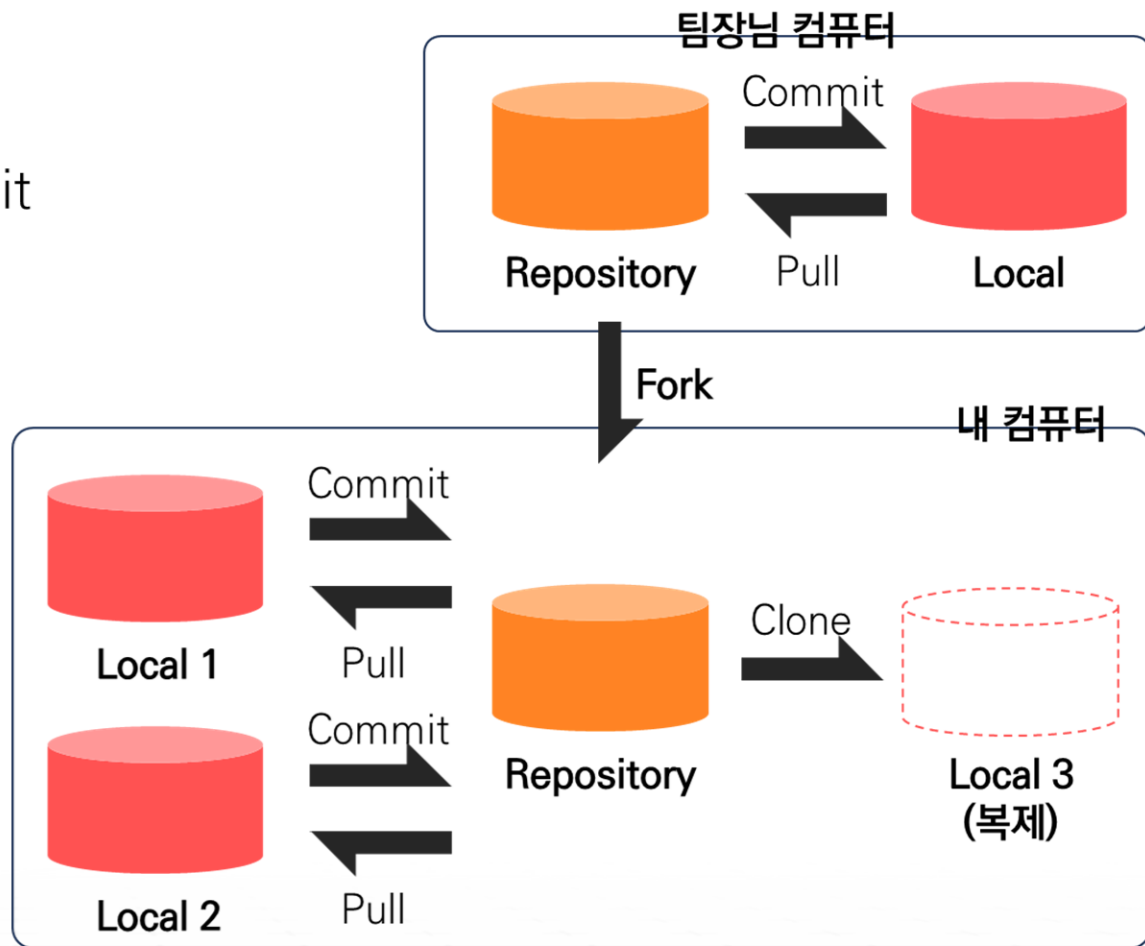


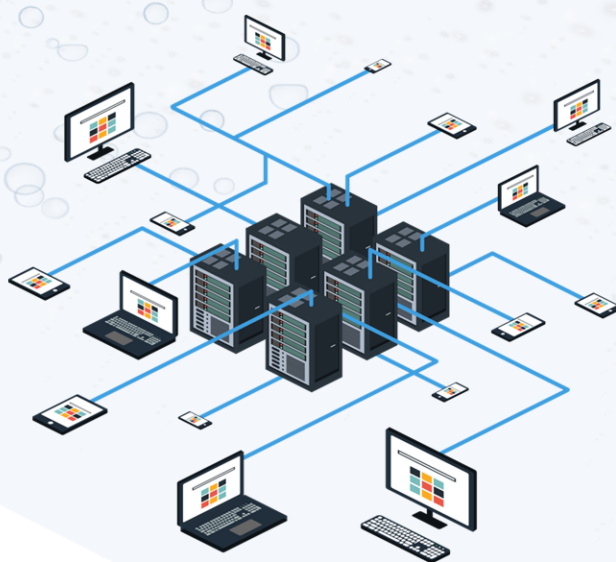


1. Git 같이 쓰기

1) Pull Request 과정

- ① `git clone https://github.com/내계정/저장소.git`
- ② `git checkout -b <새 브랜치 생성>`
- ③ 코드 수정
- ④ `git add .`
- ⑤ `git commit`
- ⑥ `git push <원격 저장소명> <브랜치명>`
 - `git push -set-upstream origin <새 브랜치명>`





Thank You

Email: qkdrk777777@naver.com